Cloud-Based Student Management System: Product Requirements Document

Oct 13, 2026 · @Devendra D

1. Overview

The Cloud Student Management System (CSMS) is a web application that runs a college's student administration (admissions, attendance, grades, fees, communication) entirely on managed cloud services, so it is reachable from any device, scales with load and needs no on-premise server.

1.1 Problem statement

Many institutions still track students in spreadsheets, paper registers and disconnected tools. This causes duplicate data, slow fee and result processing, no real-time visibility for parents, and a single point of failure when one office computer breaks. A cloud-hosted system fixes availability, backup and access, and is a good vehicle for demonstrating core cloud computing ideas: IaaS/PaaS/SaaS, elasticity, managed databases, object storage, serverless, identity management and monitoring.

1.2 Goals

- Provide one source of truth for every student record, from admission to alumni.
- Cover the full academic cycle: admissions, enrolment, timetable, attendance, assessments, results, fees, library, hostel, transport, notices and reports.
- Give each role (admin, teacher, student, parent, accountant, librarian) only the screens and data it needs.
- Run on cloud services that scale automatically, back up automatically and are secured by default.
- Be demonstrable end to end within a mini-project timeline (about 6 weeks), with a clear MVP and stretch list.

1.3 Scope

In scope	Out of scope
Web app (responsive, works on phones)	Native Android / iOS apps

In scope	Out of scope
Role-based access, 6 user roles	Integration with a real government or university portal
13 functional modules (section 3)	Biometric / face-recognition attendance hardware
Cloud deployment, storage, notifications, monitoring	Live payment settlement with real money (sandbox gateway only)
Reports and dashboards, PDF/Excel export	Multi-college (multi-tenant) billing

1.4 Success metrics

Metric	Target
Page load time (95th percentile)	under 2 seconds
API response time (95th percentile)	under 500 ms
Availability during demo and testing	99.9% or higher
Time to mark attendance for a class of 60	under 1 minute
Time to generate a report card PDF	under 5 seconds
Concurrent users supported in load test	500
Modules implemented and demonstrated	all MVP modules, at least 3 stretch features

1.5 Assumptions

- One institution (single tenant) with up to 5,000 students and 300 staff.
- Primary cloud is AWS (free tier friendly); Azure and GCP equivalents are listed in section 5 so you can switch.
- Students and parents sign in with email or phone OTP; staff sign in with email and password plus optional MFA.

2. Users and roles

The system has six roles, and every screen and API call is checked against the role, so a student can never see another student's marks and a teacher can only edit their own classes.

2.1 Personas

Role	Who they are	Main goal
Super admin	IT / principal's office	Configure the institution, manage users, see everything
Academic admin (registrar)	Admission and exam office	Admit students, build timetables, publish results
Teacher / faculty	Subject teacher, class mentor	Take attendance, enter marks, post assignments
Student	Enrolled learner	See timetable, attendance, marks, fees; submit work
Parent / guardian	Linked to one or more students	Track attendance, results and dues; get alerts
Accountant / librarian	Fee office, library staff	Collect fees and issue receipts; issue and return books

2.2 Permission matrix

C = create, R = read, U = update, D = delete, own = only their own records, class = only their assigned classes.

Module	Super admin	Academic admin	Teacher	Student	Parent	Accountant / librarian
Users and roles	CRUD	R	none	none	none	none
Admissions	CRUD	CRUD	none	none	none	R
Student profiles	CRUD	CRUD	R (class)	R, U (own, limited)	R (child)	R
Attendance	CRUD	R	CRU (class)	R (own)	R (child)	none
Timetable	CRUD	CRUD	R (own)	R (own)	R (child)	none
Courses and syllabus	CRUD	CRUD	RU (own)	R	R	none
Assignments	R	R	CRUD (class)	R, submit (own)	R (child)	none
Exams and marks	CRUD	CRUD	CRU (class)	R (own)	R (child)	none
Fees and payments	CRUD	R	none	R, pay (own)	R, pay (child)	CRUD

Module	Super admin	Academic admin	Teacher	Student	Parent	Accountant / librarian
Library	R	R	R	R, reserve	R	CRUD
Hostel and transport	CRUD	CRUD	none	R (own)	R (child)	R
Notices and messages	CRUD	CRUD	CRU (class)	R	R	R
Reports and analytics	all	academic	class only	own only	child only	finance / library only

2.3 Key user stories

- As a teacher, I mark attendance for my class in under a minute and absent students' parents are notified automatically.
- As a student, I see my timetable, attendance percentage, internal marks and pending fees on one dashboard.
- As a parent, I get an SMS or email when my child is absent or a fee is due.
- As a registrar, I accept an online application, verify documents and generate an enrolment number.
- As an accountant, I record a payment and the system issues a numbered PDF receipt.
- As an admin, I download a monthly report of attendance, fee collection and pass percentage.

3. Functional requirements

The system has 13 modules. Each requirement is tagged MVP (must be done for the demo) or Stretch (do if time allows).

3.1 Authentication and user management

ID	Requirement	Priority
AUTH-1	Sign up and sign in with email and password; email verification	MVP
AUTH-2	Role-based access control with the six roles in section 2	MVP
AUTH-3	Forgot / reset password through an emailed link	MVP
AUTH-4	Multi-factor authentication (OTP) for admin and staff	Stretch

ID	Requirement	Priority
AUTH-5	Admin can create, deactivate and bulk-import users from CSV	MVP
AUTH-6	Audit log of logins and sensitive changes (who, what, when, IP)	Stretch

3.2 Admissions and enrolment

ID	Requirement	Priority
ADM-1	Public online application form (personal, guardian, previous marks)	MVP
ADM-2	Upload documents (photo, ID, mark sheets) with file type and size checks	MVP
ADM-3	Application status flow: submitted, under review, accepted, rejected, enrolled	MVP
ADM-4	Registrar reviews, comments and approves; auto-generates enrolment number	MVP
ADM-5	Email / SMS notification on every status change	MVP
ADM-6	Merit list generation and seat allocation by course	Stretch

3.3 Student profile and records

ID	Requirement	Priority
STU-1	Profile with personal details, guardian details, address, photo, blood group	MVP
STU-2	Academic details: programme, batch, semester, section, roll number	MVP
STU-3	Document locker for ID proofs, certificates and transfer papers	MVP
STU-4	Search and filter students by name, roll number, class, status	MVP
STU-5	Bulk import and export of student data (CSV / Excel)	MVP

ID	Requirement	Priority
STU-6	Student life-cycle states: active, on leave, graduated, alumni, dropped	Stretch
STU-7	Disciplinary and achievement records	Stretch

3.4 Courses, curriculum and timetable

ID	Requirement	Priority
CRS-1	Manage departments, programmes, semesters and subjects with credits	MVP
CRS-2	Assign teachers to subjects and classes	MVP
CRS-3	Student course enrolment per semester (core and elective)	MVP
CRS-4	Weekly timetable builder with clash detection for teacher, room and class	MVP
CRS-5	Syllabus and study material upload per subject	MVP
CRS-6	Academic calendar with holidays, exam dates and events	Stretch

3.5 Attendance

ID	Requirement	Priority
ATT-1	Teacher marks present / absent / late per period for a class	MVP
ATT-2	Edit window: teacher can correct within 24 hours, admin any time with a reason	MVP
ATT-3	Attendance percentage per student and subject, with a below-75% warning	MVP
ATT-4	Automatic SMS or email to parents when a student is absent	MVP
ATT-5	Leave application by student or parent; teacher approves or rejects	Stretch

ID	Requirement	Priority
ATT-6	QR-code based attendance (student scans a code shown in class)	Stretch

3.6 Assignments and learning material

ID	Requirement	Priority
ASG-1	Teacher creates assignment with description, due date and attachment	MVP
ASG-2	Student submits file before the deadline; late submissions flagged	MVP
ASG-3	Teacher grades submission with marks and feedback	MVP
ASG-4	Reminder notification 24 hours before the due date	Stretch
ASG-5	Plagiarism check or similarity score on text submissions	Stretch

3.7 Examinations, marks and results

ID	Requirement	Priority
EXM-1	Define exams (internal, mid-term, final) with subject, date, max marks and weightage	MVP
EXM-2	Teacher enters marks per student; validation against max marks	MVP
EXM-3	Automatic grade, grade point and SGPA / CGPA calculation	MVP
EXM-4	Admin publishes results; students see them only after publishing	MVP
EXM-5	Report card and marksheet as downloadable PDF	MVP
EXM-6	Hall ticket generation and exam seating plan	Stretch
EXM-7	Revaluation request and tracking	Stretch

3.8 Fees and payments

ID	Requirement	Priority
FEE-1	Fee structure per programme and semester (tuition, lab, exam, hostel, transport)	MVP
FEE-2	Fee dues per student with due date and late fine rule	MVP
FEE-3	Online payment through a sandbox gateway (Razorpay test mode or Stripe test mode)	MVP
FEE-4	Offline payment entry by accountant (cash, cheque, UPI reference)	MVP
FEE-5	Numbered PDF receipt emailed automatically	MVP
FEE-6	Scholarships, concessions and refunds	Stretch
FEE-7	Defaulter list and automatic due reminders	Stretch

3.9 Library

ID	Requirement	Priority
LIB-1	Book catalogue with ISBN, author, category, copies available	MVP
LIB-2	Issue and return with due date; fine for overdue books	MVP
LIB-3	Search catalogue by title, author, ISBN	MVP
LIB-4	Reserve a book that is currently issued	Stretch
LIB-5	Digital library: upload and read e-books and previous question papers	Stretch

3.10 Hostel and transport

ID	Requirement	Priority
HOS-1	Hostel blocks, rooms and bed capacity	Stretch
HOS-2	Allocate and vacate rooms; room change request	Stretch

ID	Requirement	Priority
HOS-3	Hostel fee and mess charges linked to the fee module	Stretch
TRN-1	Bus routes, stops, vehicles and drivers	Stretch
TRN-2	Assign students to routes; transport fee linked to fee module	Stretch

3.11 Communication and notifications

ID	Requirement	Priority
COM-1	Notice board: admin and teachers post notices to all, a department or a class	MVP
COM-2	Email notifications for admission, attendance, results and fees	MVP
COM-3	SMS notifications for absence and fee due	Stretch
COM-4	In-app notification bell with read / unread state	MVP
COM-5	Teacher to parent messaging and a basic query / helpdesk ticket	Stretch
COM-6	Event calendar with RSVP	Stretch

3.12 Dashboards, reports and analytics

ID	Requirement	Priority
RPT-1	Role-specific home dashboard (student, teacher, parent, admin)	MVP
RPT-2	Attendance report by class, student, date range	MVP
RPT-3	Result analysis: pass percentage, toppers, subject-wise averages	MVP
RPT-4	Fee collection and outstanding dues report	MVP
RPT-5	Export any report to PDF and Excel / CSV	MVP
RPT-6	Charts for trends (attendance over months, fee collected per month)	Stretch

ID	Requirement	Priority
RPT-7	At-risk student flag (low attendance plus falling marks)	Stretch

3.13 Settings and administration

- Institution profile: name, logo, address, academic year and term dates (MVP).
- Master data: departments, programmes, grading scale, fee heads, holidays (MVP).
- Role and permission management and activity log (MVP / Stretch).
- Data backup status and one-click export of all data (Stretch).

4. Non-functional requirements

The cloud-specific qualities below are what separate this project from an ordinary college web app, so each one maps to a concrete cloud feature you can demonstrate.

Area	Requirement	How the cloud delivers it
Scalability	Handle 500 concurrent users and scale out automatically under load	Auto Scaling group or serverless functions behind a load balancer
Availability	99.9% uptime; survive one availability zone failing	Multi-AZ database, load balancer health checks
Performance	Pages under 2 s, APIs under 500 ms (p95)	CDN for static files, database indexes, caching
Security	HTTPS everywhere, encrypted data, least-privilege access	TLS certificates, encryption at rest (KMS), IAM roles, security groups
Authentication	Secure sign-in, token-based sessions, optional MFA	Managed identity service (Cognito / Azure AD B2C / Firebase Auth)
Data protection	Student data and documents never public	Private storage buckets, pre-signed URLs with short expiry
Backup and recovery	Daily backups, recovery point 24 h, recovery time 4 h	Automated database snapshots, storage versioning
Reliability	Failed notifications and jobs are retried, not lost	Message queue with dead-letter queue

Area	Requirement	How the cloud delivers it
Observability	Logs, metrics and alerts for errors and slow requests	Centralised logging, dashboards, alarms
Maintainability	Modular code, API documentation, automated deploys	Git, CI/CD pipeline, infrastructure as code
Usability	Responsive on mobile, accessible (WCAG 2.1 AA basics)	Frontend framework, CDN delivery
Compliance	Respect student privacy (India DPDP Act 2023 principles) and keep an audit trail	Role-based access, audit logs, data export and deletion on request
Cost	Stay inside free tier or under about 1,500 INR per month for the demo	Pay-per-use services, budget alerts, shut down idle resources

4.1 Security checklist

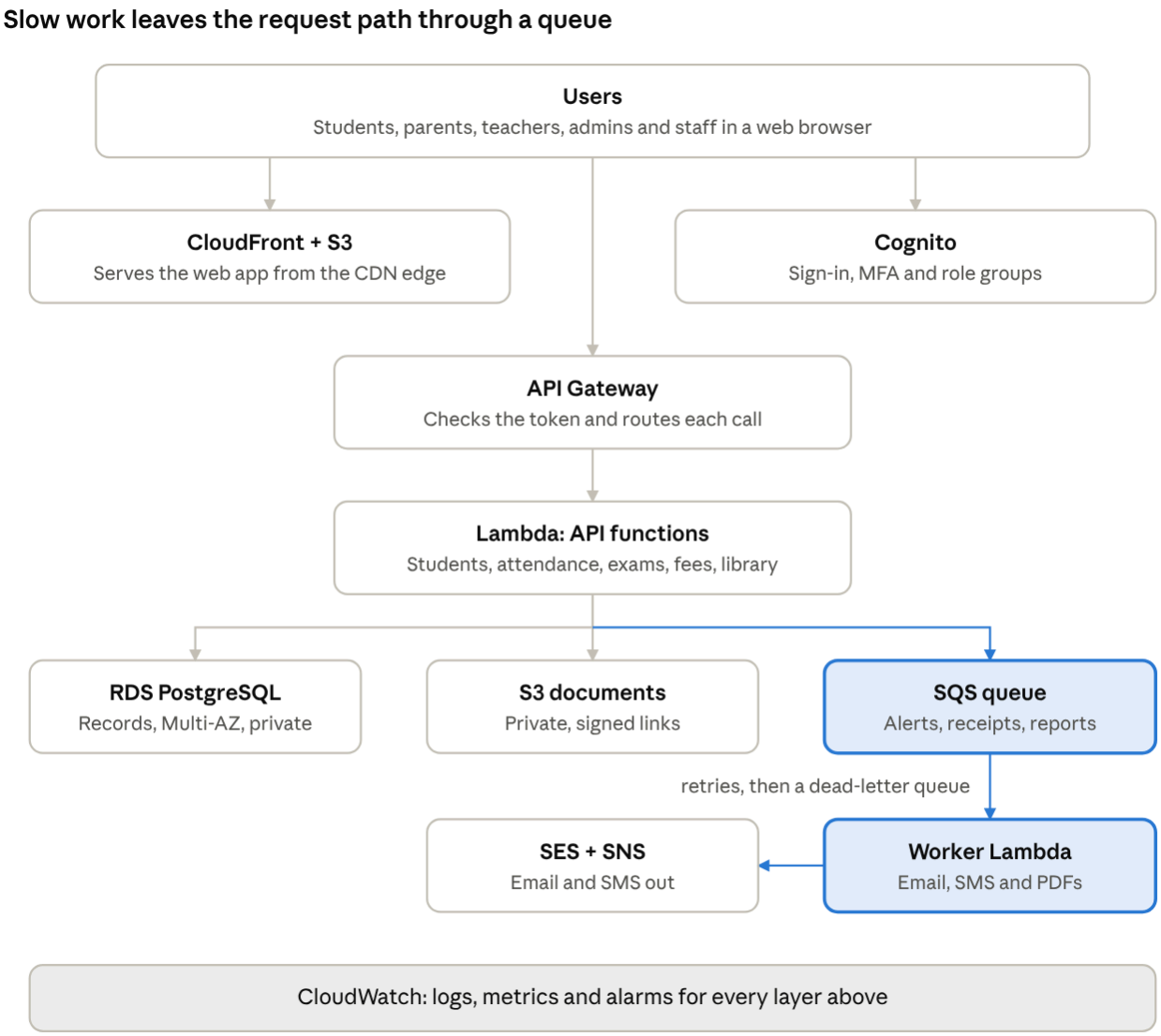
- Passwords are never stored by the app; the identity service holds them hashed.
- Every API call carries a signed token; the backend checks the role before touching data.
- Database sits in a private subnet and is not reachable from the internet.
- Secrets (database password, payment keys) live in a secrets manager, never in code.
- Uploads are type- and size-checked and virus-scanned (stretch).
- Rate limiting and a web application firewall block abuse (stretch).

5. Cloud architecture

CSMS uses a serverless-first, three-tier design on AWS: a static frontend on a CDN, a stateless API on Lambda, and managed data services, with queues carrying slow work such as notifications and PDFs away from the user's request.

5.1 Architecture diagram

The diagram shows how a request travels from the browser through the CDN, API gateway and functions to the database, storage and the notification queue.



AWS architecture · 10 services, 1 asynchronous path

The highlighted boxes are the asynchronous path: the API saves the record, drops a message on the queue and returns at once, while the worker sends email, SMS and PDFs.

5.2 Service mapping

Need	AWS service	Azure equivalent	GCP equivalent	Cloud model
Host the web app	S3 + CloudFront	Blob Storage static site + Front Door	Cloud Storage + Cloud CDN	SaaS / PaaS
Run business logic	Lambda (or EC2 / Elastic Beanstalk)	Functions (or App Service)	Cloud Functions / Cloud Run	FaaS / PaaS
Public API entry	API Gateway	API Management	API Gateway	PaaS

Need	AWS service	Azure equivalent	GCP equivalent	Cloud model
Relational data	RDS PostgreSQL	Azure Database for PostgreSQL	Cloud SQL	PaaS (DBaaS)
Files and documents	S3	Blob Storage	Cloud Storage	IaaS / PaaS
Sign-in and roles	Cognito	Entra External ID (B2C)	Identity Platform / Firebase Auth	SaaS
Background jobs	SQS + Lambda	Service Bus + Functions	Pub/Sub + Cloud Functions	PaaS
Email and SMS	SES + SNS	Communication Services	Firebase / third-party (Twilio)	SaaS
Cache	ElastiCache (Redis)	Azure Cache for Redis	Memorystore	PaaS
Scheduled jobs (nightly summaries, fee reminders)	EventBridge Scheduler	Logic Apps / Timer trigger	Cloud Scheduler	PaaS
Secrets and keys	Secrets Manager + KMS	Key Vault	Secret Manager + Cloud KMS	PaaS
Logs and alarms	CloudWatch	Azure Monitor	Cloud Monitoring	PaaS
Network security	VPC, security groups, WAF	VNet, NSG, WAF	VPC, firewall rules, Cloud Armor	IaaS
Deployments	GitHub Actions + SAM / Terraform	Azure DevOps / GitHub Actions	Cloud Build	DevOps

5.3 Deployment model

- Public cloud, single region (Mumbai, ap-south-1, for low latency to Indian users), with the database spread across two availability zones in production.
- Lambda functions run in a private subnet to reach the database; the database accepts connections only from them.

- Static files are cached at CDN edge locations; documents are served by short-lived signed links.
- Optional stretch: a second region with database read replica for disaster recovery.

6. Data model

Student data is highly relational (a student belongs to a class, a class has subjects, a subject has exams), so the primary store is a managed relational database (PostgreSQL or MySQL on Amazon RDS). Files go to object storage and only their keys are stored in the database.

6.1 Core entities

Entity	Key fields	Relationships
users	id, email, role, status, created_at	one-to-one with student, teacher or parent profile
students	id, enrolment_no, name, dob, gender, phone, address, programme_id, batch, semester, section, status	belongs to programme; many-to-many with parents and courses
parents	id, user_id, name, phone, relation	linked to one or more students
teachers	id, user_id, name, department_id, designation, phone	teaches many subjects
departments	id, name, head_teacher_id	has many programmes and teachers
programmes	id, name, department_id, duration_semesters	has many subjects and students
subjects	id, code, name, credits, semester, programme_id, type (core / elective)	taught by teachers; has exams
classes (sections)	id, programme_id, semester, section, academic_year	has students and timetable slots
enrolments	id, student_id, subject_id, academic_year, status	links students to subjects
timetable_slots	id, class_id, subject_id, teacher_id, room, day, start_time, end_time	unique per teacher, room and class at a time

Entity	Key fields	Relationships
attendance	id, student_id, subject_id, date, period, status, marked_by	one row per student per period
assignments	id, subject_id, class_id, title, description, due_at, file_key	has many submissions
submissions	id, assignment_id, student_id, file_key, submitted_at, marks, feedback	one per student per assignment
exams	id, subject_id, type, date, max_marks, weightage, published	has many marks
marks	id, exam_id, student_id, marks_obtained, grade	one per student per exam
fee_structures	id, programme_id, semester, head, amount, due_date	used to generate student dues
fee_dues	id, student_id, fee_structure_id, amount, fine, status	has many payments
payments	id, due_id, amount, mode, gateway_ref, paid_at, receipt_no	one receipt per payment
books	id, isbn, title, author, category, copies_total, copies_available	has many issues
book_issues	id, book_id, student_id, issued_on, due_on, returned_on, fine	links books and students
hostel_rooms / allocations	id, block, room_no, capacity; student_id, room_id, from, to	student to room
routes / assignments	id, name, stops, fee; student_id, route_id	student to bus route
notices	id, title, body, audience (all / department / class), created_by, created_at	shown to matching users
notifications	id, user_id, channel, message, status, sent_at, read_at	one per message per user
applications	id, applicant_name, programme_id, status, documents, reviewed_by	becomes a student on enrolment

Entity	Key fields	Relationships
audit_logs	id, user_id, action, entity, entity_id, ip, created_at	append only

6.2 Design notes

- Use UUID primary keys and foreign keys with indexes on student_id, class_id, date and status.
- Add a unique constraint on (student_id, subject_id, date, period) in attendance to stop double marking.
- Soft delete (a deleted_at column) for students and users so history is preserved.
- Store money as integers in paise, never floating point numbers.
- Cache dashboard summaries (for example attendance percentage) in Redis (ElastiCache) or a summary table, refreshed by a nightly job.
- Store documents, photos and receipts in S3 under paths like `students/{id}/documents/` and keep only the key in the database.

7. API design and key flows

The backend exposes a REST API under `/api/v1`, with JSON bodies and a bearer token on every call except sign-in and the public admission form. List endpoints support `?page=`, `?limit=` and filters.

7.1 Main endpoints

Area	Endpoints
Auth	POST /auth/login, POST /auth/refresh, POST /auth/forgot-password
Users	GET/POST /users, PATCH /users/{id}, POST /users/import
Admissions	POST /applications (public), GET /applications, PATCH /applications/{id}/status
Students	GET/POST /students, GET/PATCH /students/{id}, POST /students/import, GET /students/{id}/documents

Area	Endpoints
Courses	GET/POST /subjects, POST /enrolments, GET /timetable?classId=
Attendance	POST /attendance (bulk for a class), GET /attendance?studentId=&from=&to=, GET /attendance/summary
Assignments	POST /assignments, POST /assignments/{id}/submissions, PATCH /submissions/{id}/grade
Exams	POST /exams, PUT /exams/{id}/marks, POST /exams/{id}/publish, GET /students/{id}/report-card
Fees	GET /fees/dues?studentId=, POST /payments/initiate, POST /payments/webhook, GET /payments/{id}/receipt
Library	GET /books, POST /book-issues, PATCH /book-issues/{id}/return
Notices	GET/POST /notices, GET /notifications, PATCH /notifications/{id}/read
Files	POST /files/upload-url (returns a pre-signed URL), GET /files/{key}/download-url
Reports	GET /reports/attendance, /reports/results, /reports/fees, with `?format=pdf` or `xlsx`

7.2 Key flows

Flow 1: marking attendance and alerting parents

1. Teacher opens a class and submits the present / absent list.
2. The API saves the rows in one transaction and returns success.
3. An event is placed on a queue for each absent student.
4. A worker function reads the queue, looks up the parent's contact and sends an email (and SMS).
5. The result is saved in notifications; failures retry three times, then go to a dead-letter queue.

Flow 2: uploading a document securely

1. The browser asks the API for an upload URL.
2. The API checks the user's role and returns a pre-signed URL valid for 5 minutes.
3. The browser uploads straight to the storage bucket, so large files never pass through the server.
4. A storage event triggers a function that validates the file and saves its key in the database.

Flow 3: paying a fee

1. Student opens dues and chooses Pay now; the API creates an order with the sandbox gateway.
2. The student pays on the gateway's page.
3. The gateway calls the payment webhook; the API verifies the signature and marks the due as paid.
4. A function generates a numbered PDF receipt, stores it in the bucket and emails it.

Flow 4: publishing results

1. Teachers enter marks; the registrar checks them and clicks Publish.
2. A job calculates grades, SGPA and CGPA for every student and saves them.
3. Notifications go to students and parents; report-card PDFs are generated on demand and cached.

8. Tech stack, DevOps and cost

Pick one stack and stay with it; the suggestion below is the quickest to finish and deploy for a mini project.

8.1 Recommended stack

Layer	Choice	Alternative
Frontend	React with Vite, Tailwind CSS, hosted on S3 + CloudFront	Angular, Next.js
Backend	Node.js (Express or NestJS) on AWS Lambda behind API Gateway	Python Flask / FastAPI, Spring Boot on EC2 or Elastic Beanstalk
Database	PostgreSQL on Amazon RDS (db.t3.micro)	MySQL, DynamoDB for notifications only
Auth	Amazon Cognito user pools with groups for roles	Firebase Auth, Auth0

Layer	Choice	Alternative
Files	Amazon S3 with pre-signed URLs	Azure Blob, Google Cloud Storage
Messaging	SQS queue + SNS / SES for email and SMS	RabbitMQ, SendGrid
Reports / PDF	Lambda function using a PDF library (pdfkit, puppeteer)	Server-side templates
Payments	Razorpay or Stripe test mode	Mock payment screen

8.2 DevOps and operations

- Source control: GitHub, with `main` and feature branches and pull requests.
- CI/CD: GitHub Actions runs lint and tests, builds the frontend, then deploys to S3 and updates Lambda on every merge to main.
- Infrastructure as code: AWS SAM, Terraform or CloudFormation so the whole environment can be recreated with one command.
- Environments: dev and prod, each with its own database and bucket.
- Monitoring: CloudWatch logs, dashboard and alarms for 5xx errors, high latency and database CPU; X-Ray for tracing (stretch).
- Testing: unit tests (Jest), API tests (Postman collection) and a load test (k6 or JMeter) for the scalability demo.

8.3 Estimated monthly cost for the demo

Figures are approximate and depend on region and free-tier eligibility; check the AWS pricing calculator before deploying.

Service	Usage in demo	Approx. cost (USD / month)
Lambda + API Gateway	under 1 million requests	0 (free tier)
RDS PostgreSQL db.t3.micro	single AZ, 20 GB	0 for first 12 months, about 15 after
S3 + CloudFront	under 5 GB, light traffic	0 to 1
Cognito	under 50,000 monthly users	0

Service	Usage in demo	Approx. cost (USD / month)
SQS, SNS, SES	a few thousand messages	0 to 1
CloudWatch	basic logs and alarms	0 to 3
Total		about 0 to 5 during free tier, about 20 to 25 afterwards

Set an AWS budget alert at 5 USD, and delete or stop the database and load balancer after your demo.

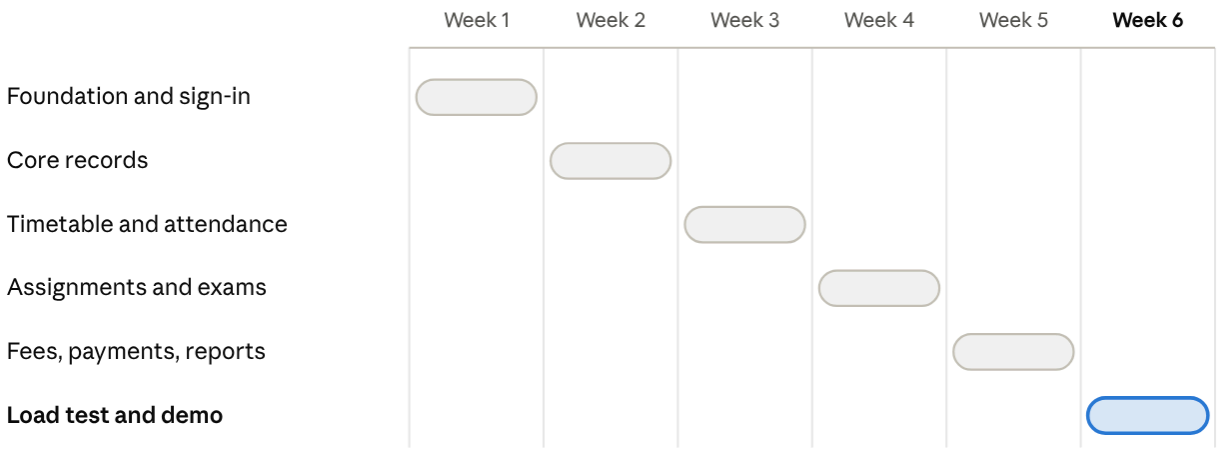
9. Delivery plan

Build the MVP in six weeks, one vertical slice at a time, so something demonstrable exists from week 2 onward.

9.1 Six-week timeline

Weeks 1 to 2 set up the cloud and the core records, weeks 3 to 4 add the daily academic features, and weeks 5 to 6 add money, reports, hardening and the demo.

Six one-week phases build up to the demo in week 6



delivery plan · 6 phases over 6 weeks

Each phase ends with something demonstrable; the last week is reserved for load testing, hardening and demo rehearsal.

9.2 Phase deliverables

Week	Focus	Deliverables
1	Foundation	Requirements signed off, cloud account and budget alert, repository, CI/CD skeleton, database schema, Cognito sign-in
2	Core records	Student, teacher, department, programme and course modules; user import; role-based screens
3	Daily academics	Timetable with clash check, attendance with parent alerts via queue, notice board
4	Assessment	Assignments with file upload, exams and marks, grade calculation, report card PDF
5	Money and reports	Fee structure, sandbox payment, receipts, dashboards, attendance / result / fee reports
6	Harden and demo	Load test, security review, monitoring dashboard, stretch features, documentation, demo rehearsal

9.3 Risks and fallbacks

Risk	Likelihood	Mitigation
Cloud cost overrun	Medium	Budget alert at 5 USD, free-tier services, stop idle resources
Too many modules for the time	High	Build all MVP items first; hostel, transport and QR attendance are explicitly stretch
Payment gateway approval or setup delay	Medium	Use test mode, or a mock payment screen that calls the same webhook
Free-tier limits or account issues	Low	Keep Azure or GCP equivalents ready (section 5); keep deployment scripted
Data loss or misconfiguration during demo	Low	Automated snapshots, seed-data script to rebuild demo data quickly
Team members blocked on one another	Medium	Agree on the API contract in week 1 and use mock responses until endpoints are ready

9.4 Testing strategy

- Unit tests for grade calculation, fine calculation and attendance percentage.
- API tests for every endpoint, including permission failures (a student calling a teacher endpoint must get 403).
- End-to-end tests for the four key flows in section 7.
- Load test with 500 virtual users on login, dashboard and attendance submission; record response times and the autoscaling behaviour.
- Security checks: no public bucket, no database access from the internet, no secrets in the repository.

9.5 Demo checklist

- [ ] Seed data loaded: 200 students, 10 teachers, 3 programmes, 1 semester of timetable.
- [ ] Show sign-in as each role and the different dashboards.
- [ ] Mark attendance and show the parent email arriving through the queue.
- [ ] Upload a document and show it stored privately in the bucket.
- [ ] Pay a fee in sandbox mode and show the receipt PDF.
- [ ] Publish results and open a report card.
- [ ] Run the load test and show autoscaling, logs and the monitoring dashboard.
- [ ] Show the architecture diagram and the pipeline that deployed it.
- [ ] Show the cost dashboard and explain how each cloud service maps to a requirement.